

Understanding ACT: Loss and training

Explaining the Style Variable Generation

1 The Dual Loss Objective

The complete training of ACT relies on two mathematical penalties working in tandem. The model must simultaneously become a precise physical imitator and an efficient style summarizer. The objective function evaluated during each step is defined as:

$$\mathcal{L}_{total} = \mathcal{L}_{reconst} + \beta \mathcal{L}_{KL}$$

1.1 L1 Reconstruction Loss ($\mathcal{L}_{reconst}$) vs. MSE

The primary goal of the Policy Decoder is to execute actions that match the human expert's original demonstration.

A Note on the Paper's Typo: If you look closely at the original ACT paper, Algorithm 1 (Line 9) states that $\mathcal{L}_{reconst}$ is calculated using MSE (Mean Squared Error, or L2 Loss). However, this is a famous typo in the pseudocode. In Section IV.C (page 6) of the text, the authors explicitly correct this: *"We use L1 loss for reconstruction instead of the more common L2 loss: we noted that L1 loss leads to more precise modeling of the action sequence."*

To calculate this, the model compares the predicted action chunk ($\hat{a}_{t:t+k}$) against the ground-truth human chunk ($a_{t:t+k}$). Both of these are tensors of shape `[k, action_dim]`. The L1 loss computes the absolute difference element-by-element across the entire chunk:

$$\mathcal{L}_{reconst} = \text{SmoothL1}(\hat{a}, a) = \frac{1}{k \cdot d_{action}} \sum_{t=1}^k \sum_{j=1}^{d_{action}} |a_{t,j} - \hat{a}_{t,j}|$$

(Note: Implementations often use **SmoothL1Loss** quadratic close to 0 and linear after (Huber Loss) to ensure differentiability precisely at zero). The L1 Loss is deliberately chosen over the pseudocode's MSE because MSE heavily penalizes large outliers but is very forgiving of tiny errors. In physical robotics, being "slightly off" across many joints leads to catastrophic, "blurry" movements. L1 forces much tighter adherence to precise fine-grained details.

1.2 KL Divergence ($\mathcal{L}_{KL}$) and The Multi-Modal Averaging Problem

One might argue: if we just set $z = \vec{0}$ at inference time anyway, why do we bother having a style variable z during training? Why not just train a single Encoder-Decoder without the CVAE and force it to learn one single way to do the task?

The answer is the **Multi-Modal Averaging Problem**. Human demonstrations are messy and "multi-modal." Given the exact same starting position, a human might execute a task in two different ways (e.g., picking up a cup by the handle vs. from the top).

- **Without z :** A standard neural network, seeing two conflicting actions for the same image, will try to minimize the L1 Loss by *averaging* the two actions. It will steer right down the middle and crash into the side of the cup.
- **With z :** By introducing the style variable z , we give the model an "excuse" for the conflicting data. It learns that when z is positive, it should grab the handle, and when z is negative, it should grab the top. It successfully learns both methods without improperly averaging them.

1.2.1 Why the Standard Normal Prior $\mathcal{N}(0, I)$ is Required

If we allow the model to use z to separate these different "modes" (styles) during training, we still face a problem at inference: the human is no longer there to demonstrate, meaning the CVAE Encoder cannot run. We have to "guess" a z vector out of thin air.

If we did not use the KL Divergence penalty, the model would spread these different styles randomly across the infinite 512-dimensional space. Guessing a random coordinate would result in erratic, untested behavior.

By enforcing a massive KL Divergence penalty ($\beta = 10$), we violently force the model to cluster all of these different human modes tightly around the origin coordinate $(0, 0, \dots, 0)$. This mathematical constraint gives us a profound advantage for inference:

- Because we forced all valid styles to cluster around zero, setting $z = \vec{0}$ at inference time acts as a clean "tie-breaker."
- It guarantees that the Policy Decoder will execute the most robust, dominant, and "safest" average of the strategies it learned, without executing conflicting physical movements.
- The "Canonical Baseline": By penalizing large z values, the model learns to represent $z = 0$ as the absolute "standard" way of completing the task. Unique human styles are treated as small variations branching off from this golden baseline.
- Resilience: Setting $z = \vec{0}$ at inference strips away all noisy human variations and drops the robot directly onto this standard, highly-resilient baseline, guaranteeing the safest possible movement.

2 Optimization Procedure: The ADAM Optimizer

The entire architecture—including both the CVAE Encoder (parameters ϕ) and the Policy Transformer (parameters θ)—is optimized simultaneously in a single backward pass. By combining both loss constraints, the gradients flow backward from the Decoder's action outputs, through the sampled vector z , and all the way back up to the Encoder's attention layers.

To update these parameters based on the gradients, the model formally uses the **AdamW (Adaptive Moment Estimation with Weight Decay)** optimizer. AdamW is specifically chosen over standard stochastic gradient descent (SGD) because it handles the complex, noisy loss landscapes of Transformers extremely well. AdamW does three specific things:

1. **Momentum (First Moment):** It calculates an exponential moving average of the gradient. If the gradient keeps pointing in the "downward" direction across multiple batches, AdamW builds up "momentum" and takes larger steps in that direction, speeding up training.
2. **Adaptive Learning Rates (Second Moment):** Instead of using one global learning rate for all weights, AdamW calculates the moving average of the *squared* gradients. If a specific weight (like a parameter deep in the ResNet) rarely gets updated, AdamW scales up its specific learning rate. If a weight gets updated aggressively and chaotically, AdamW scales down its learning rate to prevent the model from over-correcting.
3. **Weight Decay:** A small penalty (typically $1e - 4$) is added directly to the weights themselves during the update step. This prevents any single weight from growing too large, which serves as a powerful regularization technique to prevent the network from memorizing the specific human noise in the training set (overfitting).

Typically, the ACT architecture is trained with a global learning rate of $1e - 5$ (with adaptive scaling handled internally by AdamW) spanning tens of thousands of epochs to ensure the loss smoothly converges to the optimal "canonical baseline".